

2교시: Docker bridge network와 container DNS

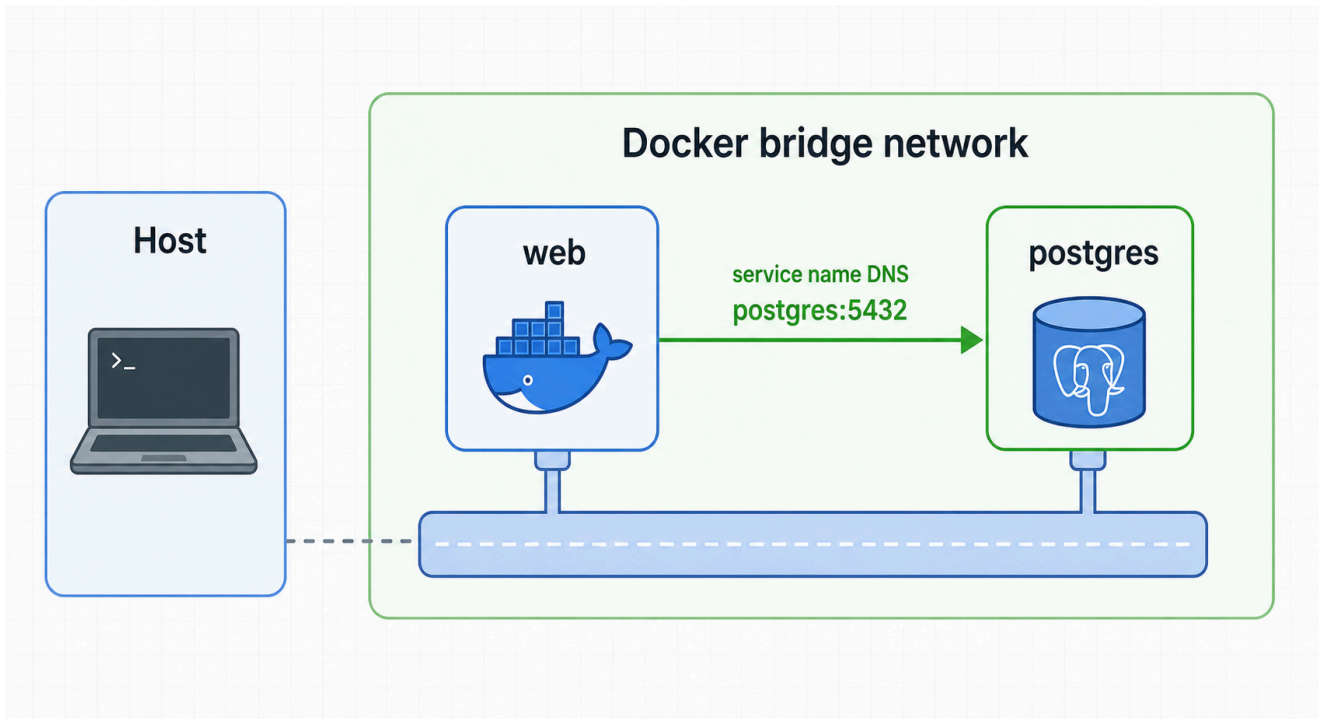
수업 목표

- default bridge와 user-defined bridge network의 차이를 설명한다.
- container끼리 같은 network에서 이름으로 접근하는 흐름을 이해한다.
- PostgreSQL container를 network에 붙이고 DNS evidence를 확보한다.

50분 흐름

시간	활동	비중	산출
0-8분	port binding 복습	설명 15%	host/container 접근 구분
8-18분	bridge network 개념	설명 20%	network note
18-32분	custom network 생성	실행 25%	network evidence
32-42분	service name DNS 확인	실행 25%	pg_isready -h output
42-50분	network 장애 분류	설명 15%	RCA note

Visual 1: Bridge network와 DNS



이 이미지는 같은 Docker bridge network 안의 container가 container name을 DNS 이름처럼 사용해 접근하는 구조를 보여준다.

핵심 설명

host에서 container로 들어갈 때는 port publishing이 필요하다. 반면 container끼리 같은 user-defined bridge network에 있으면 container name으로 서로를 찾을 수 있다. Day 4 Compose의 service name 통신은 이 개념 위에 올라간다.

paperclip-day3-net 같은 user-defined bridge network를 만들면 container를 명시적으로 같은 network에 붙일 수 있다. 이때 paperclip-day3-postgres라는 container name은 같은 network 내부에서 hostname처럼 사용할 수 있다.

container DNS는 host의 /etc/hosts를 직접 수정하는 방식이 아니다. Docker network가 container name을 resolve할 수 있게 해준다. 그래서 host에서 paperclip-day3-postgres라는 이름이 바로 통하지 않을 수 있지만, 같은 Docker network 안의 container에서는 통할 수 있다.

실행 명령

```
docker network create paperclip-day3-net

docker run -d \
  --name paperclip-day3-postgres \
  --network paperclip-day3-net \
  -e POSTGRES_PASSWORD=paperclip \
  -e POSTGRES_DB=paperclip \
  postgres:16-alpine

docker run --rm \
  --network paperclip-day3-net \
  postgres:16-alpine \
  pg_isready -h paperclip-day3-postgres -U postgres
```

Linux 사전 테스트 결과

network 생성:

```
30306ff4539250d1b6456d863487280f3f8ef2d2854b80ed80dbf24e964924bd
```

DNS/readiness 확인:

```
paperclip-day3-postgres:5432 - accepting connections
```

network inspect 핵심:

```
"Name": "paperclip-day3-postgres"
"IPv4Address": "192.168.16.2/20"
```

bridge network 해석

구분	의미
host -> container	-p host:container port publishing 필요
container -> container	같은 network 안에서는 container name 사용 가능
user-defined bridge	이름 기반 discovery와 격리 범위를 명시하기 좋음
default bridge	초급 실습에는 가능하지만 handoff가 덜 명확함
Compose network	Day 4에서 service 단위로 자동 구성됨

핵심 유의사항

container name과 image name은 다르다. postgres:16-alpine은 image name이고, paperclip-day3-postgres는 container name이다. 같은 image에서 container를 여러 개 만들 수 있지만 같은 container name은 동시에 중복할 수 없다.

같은 network에 붙어 있지 않으면 name resolution이 되지 않을 수 있다. pg_isready -h paperclip-day3-postgres가 실패하면 DB process가 죽었는지, network가 다른지, 이름이 틀렸는지를 순서대로 확인한다.

PostgreSQL port 5432가 container 내부에서 열려 있어도 host에서 접근하려면 별도 -p 15432:5432가 필요하다. Day 3 DB 실습에서는 container끼리 통신하는 구조를 보기 위해 host publish를 생략한다.

자주 놓치는 지점

놓치는 지점	증상	확인
image name으로 DNS 접근	name not found	container name 확인
network 누락	같은 이름인데 연결 실패	docker inspect, network inspect
DB readiness 전 접근	connection refused	pg_isready, logs
host와 container DNS 혼동	host terminal에서 이름 해석 실패	같은 network container에서 테스트
port publish와 network 통신 혼동	불필요한 host port 노출	internal network 사용

network evidence 명령

```
docker network inspect paperclip-day3-net
docker inspect paperclip-day3-postgres --format '{{json
.NetworkSettings.Networks}}'
docker ps --filter name=paperclip-day3-postgres
```

읽을 항목:

- network name
- container name
- IP address
- exposed container port
- host publish 여부

장애 분기표

관찰	가능성	다음 명령
No such network	network 생성 누락	docker network ls
Name already in use	container name 중복	docker ps -a
no response	DB readiness 미완료	docker logs, pg_isready
host에서 name 접근 실패	정상일 수 있음	같은 network container에서 확인
network 삭제 실패	container가 아직 연결됨	container 제거 후 삭제

운영 관점

DB처럼 내부 service는 host port를 열지 않아도 된다. 외부에서 접근해야 하는 web service만 host port를 publish하고, DB는 app container와 같은 network 안에서 service name으로 접근하는 편이 더 안전하다.

Day 4 Compose에서는 이 구조가 services.web, services.db, networks로 표현된다. Day 3에서 길게 친 docker network create와 --network 옵션을 먼저 이해해야 Compose network가 자동으로 만들어지는 이유가 보인다.

기록 템플릿

```
## Lesson 2 Network Evidence
- network name:
- DB container name:
- image:
```

- readiness command:
- readiness result:
- DNS name used:
- host port publish 여부:
- network inspect에서 확인한 container:

마무리 점검

host에서 container로 접근할 때는 ____가 필요하다.
 같은 Docker network의 container끼리는 ____으로 접근할 수 있다.
 DB container를 외부에 공개하지 않아도 되는 이유는 ____이다.

cleanup

```
docker rm -f paperclip-day3-postgres
docker network rm paperclip-day3-net
```

평가 기준

기준	2점 evidence
network	custom bridge network를 만들었다
DNS	container name으로 DB readiness를 확인했다
구분	host publish와 internal network를 구분했다
장애	network/name/readiness 실패를 분류했다

공식 근거 링크

- Docker networking: <https://docs.docker.com/engine/network/>
- Docker run reference: <https://docs.docker.com/reference/cli/docker/container/run/>